

CSS Layout

Concetti Fondamentali

Il **Layout CSS** determina come gli elementi sono posizionati e organizzati nella pagina. CSS offre diversi metodi di layout, ognuno con caratteristiche e casi d'uso specifici.

Evoluzione Storica

Periodo	Metodo	Descrizione
Anni '90	Table-based layout	Si usavano le tabelle HTML per creare colonne e strutture
Anni 2000	Float-based layout	Si usava <code>float</code> per affiancare gli elementi a colonna
Sempre presente	Positioning	Posizionamento assoluto o relativo degli elementi
2010+	Flexbox	Layout monodimensionale (righe o colonne)
2017+	Grid	Layout bidimensionale (righe e colonne insieme)

Proprietà `display`

La proprietà `display` è la base di tutti i layout CSS. Determina il **tipo di rendering box** che un elemento genera, cioè come si comporta nella pagina.

```
elemento { display: valore; }
```

`block`

Elementi che **occupano tutta la larghezza disponibile** e iniziano su una nuova riga.

```
div { display: block; }  
/* Comportamento: larghezza 100%, nuova riga */  
/* Default per: div, h1-h6, p, header, section */
```

Esempio:

```
<div>Sono un blocco</div>  
<div>Sono un altro blocco – vado a capo automaticamente</div>
```

Pensa a ogni `div` come a un mattone che occupa tutta la larghezza del muro.

inline

Elementi che si comportano **come testo**, affiancati sulla stessa riga.

```
span { display: inline; }  
/* Comportamento: larghezza del contenuto, stessa riga */  
/* Default per: span, a, strong, em, img */
```

Esempio:

```
<p>Questo è un testo con <span>parola inline</span> e <strong>un'altra</strong> sulla stessa riga.
```

Gli elementi inline non accettano `width` e `height` — si adattano al loro contenuto.

inline-block

Via di mezzo: **si posiziona inline** (affiancato) ma **accetta larghezza e altezza** come un blocco.

```
button { display: inline-block; }  
/* Comportamento: affiancato, ma con width/height controllabili */
```

Esempio:

```
<span style="display: inline-block; width: 100px; height: 50px; background: lightblue;">Box 1</span>  
<span style="display: inline-block; width: 100px; height: 50px; background: lightcoral;">Box 2</span>
```

none

L'elemento viene **nascondo completamente** — non occupa spazio nella pagina.

```
.nascondo { display: none; }  
/* L'elemento sparisce, come se non esistesse nell'HTML */
```

Diverso da `visibility: hidden` che nasconde l'elemento ma lascia lo spazio vuoto.

Riepilogo visivo

```
display: block
```

← occupa tutta la larghezza

```
display: block
```

← va a capo

```
[inline] [inline] [inline]
```

← sulla stessa riga, larghezza del contenuto

```
[inline-block] [inline-block]
```

← stessa riga, ma con width/height impostabili

Tabella di confronto rapida

Proprietà	Nuova riga?	Width/Height?	Default per
<code>block</code>	Sì	Sì	<code>div</code> , <code>p</code> , <code>h1-h6</code>
<code>inline</code>	No	No	<code>span</code> , <code>a</code> , <code>strong</code>

<code>inline-block</code>	No	Si	<code>button</code> , <code>img</code>
<code>none</code>	—	—	(nessuno)

1. Table-Based Layout (Anni '90)

Cos'è

Prima che esistesse CSS moderno, i developer usavano i **tag `<table>` di HTML** per costruire l'intera struttura della pagina: colonne, header, sidebar, footer. Non era lo scopo originale delle tabelle (che servivano per dati tabulari), ma era l'unico modo per allineare elementi affiancati.

Come funziona

Una tabella HTML divide la pagina in righe (`<tr>`) e celle (`<td>`). Ogni cella diventa una "colonna" del layout.

```
<table width="100%">
  <tr>
    <td width="20%">
      <!-- Sidebar -->
      <p>Menu</p>
      <a href="#">Home</a>
      <a href="#">Chi siamo</a>
    </td>
    <td width="80%">
      <!-- Contenuto principale -->
      <h1>Benvenuto</h1>
      <p>Questo è il contenuto della pagina.</p>
    </td>
  </tr>
</table>
```

Risultato visivo:

Menu	Benvenuto
Home	Questo è il contenuto...

| Chi siamo |

Perché non si usa più

- Le tabelle hanno **scopo semantico** (dati tabulari, tipo Excel), non strutturale
- Il codice diventa **illeggibile** con layout complessi (tabelle dentro tabelle)
- **Pessimo per l'accessibilità** — gli screen reader leggono le tabelle come dati
- **Difficile da mantenere** e non responsive

Oggi le tabelle si usano solo per dati tabulari (prezzi, orari, confronti). Mai per il layout.

2. Float-Based Layout (Anni 2000)

Cos'è

`float` nasce per far **scorrere il testo attorno alle immagini** (come nei giornali). I developer degli anni 2000 lo "hackerano" per creare colonne affiancate.

La proprietà `float`

```
elemento { float: left | right | none; }
```

Un elemento con `float` viene **tolto dal normale flusso** della pagina e spostato a sinistra o destra. Gli altri elementi gli scorrono attorno.

Esempio: layout a due colonne

```
<div class="sidebar">Sidebar</div>
<div class="contenuto">Contenuto principale</div>
<div class="footer">Footer</div>
```

```

.sidebar {
  float: left;
  width: 25%;
  background: lightgray;
}

.contenuto {
  float: left;
  width: 75%;
  background: lightyellow;
}

.footer {
  clear: both; /* fondamentale! */
  background: lightblue;
}

```

Risultato visivo:

Sidebar (25%)	Contenuto principale (75%)
Footer (clear: both)	

Il problema del `clear`

Quando tutti i figli di un contenitore hanno `float`, il contenitore **collassa a altezza 0** perché i float sono fuori dal flusso.

```

<div class="container">
  <div style="float: left;">Box 1</div>
  <div style="float: left;">Box 2</div>
  <!-- Il container non "vede" i figli floattati → altezza 0! -->
</div>

```

Soluzione classica — il clearfix:

```
.container::after {  
  content: "";  
  display: block;  
  clear: both;  
}
```

Perché non si usa più per il layout

- Richiede **hack** come il clearfix
- Comportamento **imprevedibile** con altezze diverse tra colonne
- Difficile da centrare verticalmente
- Superato da Flexbox e Grid

float è ancora utile oggi, ma solo per il suo scopo originale: far scorrere testo attorno a un'immagine.

```
img { float: left; margin-right: 16px; }
```

3. Positioning

Cos'è

Il **positioning** permette di controllare con precisione dove un elemento appare nella pagina, uscendo dal normale flusso del documento. Si usa la proprietà `position` insieme a `top`, `right`, `bottom`, `left`.

```
elemento {  
  position: static | relative | absolute | fixed | sticky;  
  top: 20px;  
  left: 50px;  
}
```

position: static (default)

Il comportamento normale di ogni elemento. Segue il flusso del documento, le proprietà `top/left/right/bottom` non hanno effetto.

```
div { position: static; } /* comportamento di default, inutile scriverlo */
```

position: relative

L'elemento rimane nel flusso, ma viene **spostato rispetto alla sua posizione originale**. Lo spazio originale viene mantenuto.

```
.box {  
  position: relative;  
  top: 20px; /* scende di 20px dalla sua posizione naturale */  
  left: 30px; /* si sposta di 30px a destra */  
}
```

[spazio originale rimane qui]



← spostato di 20px giù e 30px a destra

relative si usa spesso come **ancora** per i figli *absolute*.

position: absolute

L'elemento viene **tolto dal flusso** e posizionato rispetto al **primo antenato con position non-static**. Se nessun antenato ha `position`, si posiziona rispetto alla pagina intera (`<html>`).

```
<div class="contenitore">  
  <div class="badge">NUOVO</div>  
    
</div>
```



```
.contenitore {
  position: relative; /* ancora per il figlio absolute */
}

.badge {
  position: absolute;
  top: 10px;
  right: 10px;
  background: red;
  color: white;
  padding: 4px 8px;
}
```



position: fixed

L'elemento viene **tolto dal flusso** e posizionato rispetto alla **viewport** (la finestra del browser). Rimane **fisso anche durante lo scroll**.

```
.navbar {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  background: white;
  box-shadow: 0 2px 4px rgba(0,0,0,0.2);
}
```

Classico caso d'uso: **navbar fissa** in cima alla pagina.

position: sticky

Ibrido tra `relative` e `fixed` . L'elemento **scorre normalmente** fino a raggiungere una soglia, poi **si blocca** in quella posizione durante lo scroll.

```
.intestazione-tabella {
  position: sticky;
  top: 0; /* si blocca quando raggiunge il top della viewport */
  background: white;
}
```

Classico caso d'uso: **header di tabella** che rimane visibile durante lo scroll verticale.

Riepilogo Positioning

Valore	Nel flusso?	Riferimento	Caso d'uso
<code>static</code>	Si	—	Default, nessuno
<code>relative</code>	Si (spazio mantenuto)	Se stesso	Ancora per figli, piccoli aggiustamenti
<code>absolute</code>	No	Antenato <code>relative</code>	Badge, tooltip, dropdown
<code>fixed</code>	No	Viewport	Navbar fissa, cookie banner
<code>sticky</code>	Si → No	Viewport (dopo soglia)	Header tabella, sidebar che segue

4. Flexbox (2010+)

Cos'è

Flexbox (Flexible Box Layout) è il primo sistema di layout CSS pensato appositamente per **distribuire e allineare elementi in una direzione** (riga o colonna).

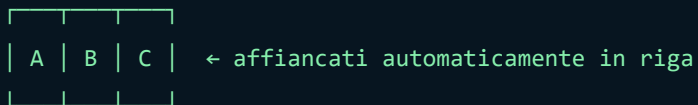
Risolve elegantemente problemi che con float erano complicati: centrare verticalmente, distribuire spazio equamente, riordinare elementi.

Concetti base

Si applica al **contenitore padre** (`display: flex`), e i figli diretti diventano automaticamente **flex items**.

```
.container {  
  display: flex;  
}
```

```
<div class="container">  
  <div class="item">A</div>  
  <div class="item">B</div>  
  <div class="item">C</div>  
</div>
```



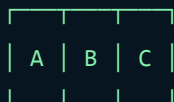
Asse principale e asse trasversale

Flexbox lavora su **due assi**:

- **Asse principale** (main axis): la direzione dei flex items
- **Asse trasversale** (cross axis): perpendicolare al principale

```
flex-direction: row (default)
```

main axis →



↑ cross axis

Proprietà del contenitore

flex-direction — direzione degli elementi

```
.container { flex-direction: row; }          /* → sinistra a destra (default) */
.container { flex-direction: row-reverse; }  /* ← destra a sinistra */
.container { flex-direction: column; }       /* ↓ dall'alto al basso */
.container { flex-direction: column-reverse; } /* ↑ dal basso all'alto */
```

justify-content — allineamento sull'asse principale

```
.container { justify-content: flex-start; } /* |ABC      | */
.container { justify-content: flex-end; }   /* |      ABC| */
.container { justify-content: center; }     /* |  ABC  | */
.container { justify-content: space-between; } /* |A      B    C| */
.container { justify-content: space-around; } /* | A      B    C | */
.container { justify-content: space-evenly; } /* |  A      B    C  | */
```

align-items — allineamento sull'asse trasversale

```
.container { align-items: stretch; } /* (default) i figli si allungano */
.container { align-items: flex-start; } /* allineati in alto */
.container { align-items: flex-end; }   /* allineati in basso */
.container { align-items: center; }     /* centrati verticalmente */
```

flex-wrap — a capo o no

```
.container { flex-wrap: nowrap; } /* (default) tutto su una riga, anche se straborda */
.container { flex-wrap: wrap; }   /* va a capo se non c'è spazio */
```

gap — spazio tra gli elementi

```
.container { gap: 16px; } /* 16px tra righe e colonne */
.container { gap: 8px 16px; } /* 8px righe, 16px colonne */
```

Proprietà dei figli (flex items)

flex — come cresce/shrink un elemento

```
.item { flex: 1; }    /* occupa lo spazio disponibile in modo equo */  
.item { flex: 2; }    /* occupa il doppio rispetto a flex: 1 */
```

```
.item-a { flex: 1 }   .item-b { flex: 2 }   .item-c { flex: 1 }
```

25%	50%	25%

align-self — sovrascrive align-items per un singolo figlio

```
.item-speciale { align-self: flex-end; }
```

order — riordina visivamente senza cambiare l'HTML

```
.item-a { order: 2; }  
.item-b { order: 1; } /* appare prima visivamente */  
.item-c { order: 3; }
```

Esempio completo: navbar

```
<nav class="navbar">  
  <div class="logo">MyBrand</div>  
  <ul class="links">  
    <li><a href="#">Home</a></li>  
    <li><a href="#">About</a></li>  
    <li><a href="#">Contact</a></li>  
  </ul>  
  <button class="cta">Accedi</button>  
</nav>
```

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 16px 32px;  
  background: #1a1a2e;  
  color: white;  
}  
  
.links {  
  display: flex;  
  gap: 24px;  
  list-style: none;  
}
```



Centrare un elemento (il problema classico!)

Con Flexbox, centrare verticalmente e orizzontalmente è banale:

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 100vh;  
}
```



5. Grid (2017+)

Cos'è

CSS Grid è il sistema di layout bidimensionale di CSS: controlla sia **righe** che **colonne** contemporaneamente. È il metodo più potente per costruire layout di pagina completi.

Mentre Flexbox eccelle per componenti lineari (navbar, card row), Grid eccelle per **layout di pagina intera**.

Concetti base

```
.container {  
  display: grid;  
  grid-template-columns: 200px 1fr 200px; /* 3 colonne */  
  grid-template-rows: 80px 1fr 60px;      /* 3 righe */  
}
```

fr = **fraction** — unità flessibile di Grid. **1fr** prende tutto lo spazio disponibile rimasto.

Definire la griglia

grid-template-columns e **grid-template-rows**

```
/* 3 colonne uguali */  
.grid { grid-template-columns: 1fr 1fr 1fr; }  
  
/* shorthand con repeat() */  
.grid { grid-template-columns: repeat(3, 1fr); }  
  
/* colonne miste */  
.grid { grid-template-columns: 250px 1fr; }  
/* sidebar fissa + contenuto flessibile */
```

gap — spazio tra le celle

```
.grid {  
  gap: 16px;      /* uguale su tutti i lati */  
  gap: 8px 16px;   /* righe, colonne */  
}
```

Posizionare gli elementi

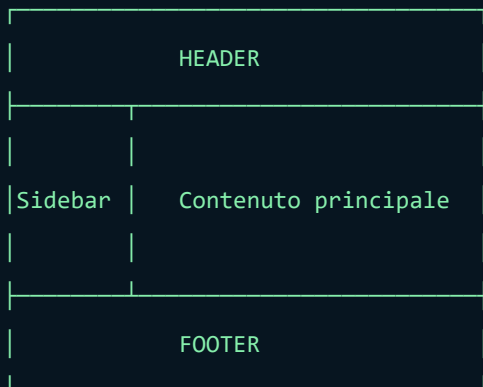
Ogni figlio può occupare più celle usando `grid-column` e `grid-row`.

Griglia 3x3 con linee numerate:

	1	2	3	4
1				
2				
3				

```
.header {  
  grid-column: 1 / 4; /* dalla linea 1 alla 4 = occupa tutte e 3 le colonne */  
  grid-row: 1 / 2;  
}  
  
.sidebar {  
  grid-column: 1 / 2;  
  grid-row: 2 / 3;  
}  
  
.main {  
  grid-column: 2 / 4;  
  grid-row: 2 / 3;  
}  
  
.footer {  
  grid-column: 1 / 4;  
  grid-row: 3 / 4;  
}
```

Risultato:



grid-template-areas — layout con nomi

Un modo più leggibile per definire il layout usando nomi invece di numeri di linea:

```
.container {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-rows: 80px 1fr 60px;  
  grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
}  
  
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.main { grid-area: main; }  
.footer { grid-area: footer; }
```

Leggendo `grid-template-areas` si "vede" il layout come una mappa. Molto intuitivo.

Griglie automatiche con `auto-fill` e `auto-fit`

Utile per **gallery di card** che si adattano automaticamente allo spazio disponibile:

```
.gallery {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));  
}
```

```
gap: 16px;
}
```

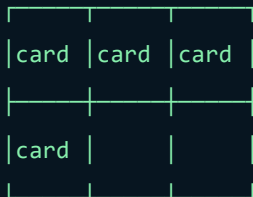
- `auto-fill` : crea quante colonne ci stanno (anche vuote)
- `auto-fit` : come `auto-fill` ma le colonne vuote collassano
- `minmax(200px, 1fr)` : ogni colonna è minimo 200px, massimo 1fr

Comportamento responsive automatico — senza media query!

Schermo largo:



Schermo medio:



Allineamento in Grid

```
/* Allinea tutti gli elementi nelle celle */
.grid {
  justify-items: start | end | center | stretch; /* orizzontale */
  align-items: start | end | center | stretch; /* verticale */
}

/* Allinea l'intera griglia nel contenitore */
.grid {
  justify-content: start | end | center | space-between;
  align-content: start | end | center | space-between;
}

/* Singolo elemento */
.item {
  justify-self: center;
  align-self: end;
}
```

Flexbox vs Grid — quando usare quale?

Situazione	Usa
Navbar, barra strumenti, row di bottoni	Flexbox
Layout di pagina (header/sidebar/main/footer)	Grid
Gallery di card responsive	Grid
Centrare un singolo elemento	Flexbox
Componenti lineari con spazio dinamico	Flexbox
Layout complessi a due dimensioni	Grid

Non si escludono: **Grid per il macro-layout, Flexbox per i componenti**. Si usano spesso insieme nello stesso progetto.

Esempio finale: pagina completa con Grid + Flexbox

```
<body>
  <header>
    <nav class="navbar">
      <span class="logo">MyApp</span>
      <ul class="nav-links">
        <li><a href="#">Home</a></li>
        <li><a href="#">About</a></li>
      </ul>
    </nav>
  </header>
  <aside>Sidebar</aside>
  <main>Contenuto</main>
  <footer>Footer</footer>
</body>
```

```
body {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
```

```
grid-template-columns: 240px 1fr;
grid-template-rows: 64px 1fr 48px;
min-height: 100vh;
}

header { grid-area: header; }
aside { grid-area: sidebar; }
main { grid-area: main; }
footer { grid-area: footer; }

/* Flexbox dentro Grid per la navbar */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 32px;
}

.nav-links {
  display: flex;
  gap: 24px;
  list-style: none;
}
```

Riepilogo generale dei metodi di layout

Metodo	Anno	Dimensioni	Oggi?	Uso consigliato
Table-based	'90	2D	No	Solo dati tabulari
Float	2000	—	Limitato	Solo testo attorno a immagini
Positioning	Sempre	—	Si	Overlay, badge, navbar fissa
Flexbox	2010	1D	Si	Componenti, allineamenti lineari
Grid	2017	2D	Si	Layout di pagina, gallery